



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de
HONORIS UNITED UNIVERSITIES

RAPPORT DE STAGE D'OBSERVATION PROFESSIONNELLE

Sujet technique :

**Conception d'une solution basée sur l'intelligence artificielle et
le machine learning
pour la supervision proactive des environnements Windows,
VMware et Hyper-V**

Stagiaire	Mehdi Lharti
Entreprise d'accueil	CGI
Encadrant CGI	Nadir Sisah
Encadrant EMSI	Jaouad Tagnamas
Nature du stage	Stage d'observation
Période	01/07/2026 – 28/08/2026
Année universitaire	2025-2026

Rapport préparé dans le respect de la confidentialité : le document ne reproduit aucune information relative à un client, aucune donnée sensible ni aucun secret opérationnel de CGI.

Remerciements

Je tiens à exprimer ma profonde gratitude à l'ensemble des personnes qui ont contribué au bon déroulement de mon stage d'observation professionnelle au sein de CGI. Cette immersion sur site m'a permis de découvrir un environnement de travail structuré, exigeant et orienté vers la qualité de service, et de confronter mes acquis académiques aux réalités d'un projet informatique à vocation professionnelle.

Je remercie tout particulièrement M. Nadir Sisah, mon encadrant au sein de CGI, pour son accompagnement, ses orientations et les échanges qui m'ont permis de mieux comprendre les attentes d'un contexte professionnel. Son suivi a constitué un repère important dans l'organisation du travail, la prise de recul sur les choix techniques et la progression du projet.

J'adresse également mes sincères remerciements à M. Jaouad Tagnamas, mon encadrant à l'EMSI, pour son encadrement académique, ses recommandations méthodologiques et l'attention portée à la cohérence entre les objectifs du stage, les apprentissages du cycle ingénieur et la rédaction de ce rapport.

Je remercie enfin les équipes de CGI pour l'accueil sur site et pour l'opportunité d'observer les pratiques d'une entreprise internationale de services numériques. Cette expérience a renforcé ma compréhension des exigences liées à l'organisation, à la sécurité, à la documentation, à la qualité logicielle, au travail en équipe et à la validation technique.

Résumé

Ce rapport présente le stage d'observation professionnelle réalisé sur site au sein de CGI. L'objectif principal du stage était de découvrir le fonctionnement d'un environnement professionnel de services numériques et de mettre en pratique, dans un cadre structuré, des compétences liées au développement logiciel, à la supervision d'infrastructures, au DevOps, à la sécurité et à l'intelligence artificielle. Le sujet technique étudié porte sur la conception d'InfraSentinel AI, une plateforme de supervision proactive destinée aux environnements Windows, VMware et Hyper-V.

La solution s'appuie sur un agent Windows, une API Django REST, PostgreSQL, Redis, Celery, un canal WebSocket en temps réel et une interface web développée avec React, Vite et TypeScript. La couche intelligente combine un moteur de règles, un modèle Isolation Forest pour la détection d'anomalies et une analyse temporelle des tendances afin d'estimer des risques avant qu'un seuil critique ne soit atteint. L'architecture a été validée dans un laboratoire local Docker avec des métriques Windows réelles et des campagnes de tests fonctionnels, de sécurité et de performance.

Les résultats obtenus montrent une plateforme stable dans les scénarios locaux validés : WebSocket opérationnel, services Docker sains, télémétrie Windows et GPU réelle, et essai local de 250 agents simulés à cadence réaliste avec 0 % d'erreur et un p95 de 98,1 ms. Une réexécution de contrôle de la version actuelle du dépôt a obtenu 63 réussites sur 64 tests de l'interface web ; le test de déconnexion d'AppShell reste à corriger. Les principales limites concernent également la validation sur des infrastructures VMware et Hyper-V réelles, certains scénarios du service Windows, l'envoi SMTP externe et la validation de la partie prédictive sur une longue durée.

Mots-clés : supervision proactive, observabilité, Windows, VMware, Hyper-V, Django, React, PostgreSQL, Redis, Celery, WebSocket, Isolation Forest, machine learning, détection d'anomalies, analyse prédictive.

Liste des sigles et abréviations

Sigle	Signification
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CPU	Central Processing Unit
GPU	Graphics Processing Unit
IA	intelligence artificielle
ML	machine learning
NOC	Network Operations Center
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
VM	Virtual Machine
VRAM	Video Random Access Memory
WebSocket	Canal bidirectionnel persistant pour les mises à jour temps réel
PFE	Projet / démonstration de fin de parcours académique dans ce contexte
ESN	Entreprise de services numériques

Liste des figures

- Figure 1 - Architecture générale d'InfraSentinel AI
- Figure 2 - Flux d'une mesure depuis la source jusqu'au tableau de bord
- Figure 3 - Architecture du laboratoire local de validation
- Figure 4 - Agent Windows de la démonstration synthétique PFE25
- Figure 5 - Vue globale de la démonstration synthétique PFE25
- Figure 6 - Inventaire des machines de la démonstration synthétique PFE25
- Figure 7 - Fiche d'une machine Windows de la démonstration synthétique PFE25
- Figure 8 - Analyse d'une anomalie de la démonstration synthétique PFE25
- Figure 9 - Historique du modèle Isolation Forest de la démonstration synthétique PFE25
- Figure 10 - Analyse prédictive de la démonstration synthétique PFE25
- Figure 11 - Vue conceptuelle des mécanismes de supervision proactive
- Figure 12 - Alerte CPU et recommandation de la démonstration synthétique PFE25

Liste des tableaux

Tableau 1 - Fiche signalétique du stage

Tableau 2 - Principaux domaines d'activité observés ou reliés au contexte du stage

Tableau 3 - Objectifs du stage

Tableau 4 - Besoins fonctionnels et non fonctionnels

Tableau 5 - Rôles et responsabilités applicatives

Tableau 6 - Technologies principales

Tableau 7 - Résultats principaux de performance après remédiation

Tableau 8 - Synthèse des validations techniques

Tableau 9 - Limites et perspectives

Table des matières

Introduction générale.....	9
Chapitre 1 - Présentation de CGI et cadre professionnel	10
1.1 Présentation générale de CGI.....	10
1.2 CGI au Maroc et site de Fès	10
1.3 Domaines d'activité et environnement professionnel.....	10
1.4 Positionnement du stage	11
Chapitre 2 - Déroulement du stage d'observation.....	12
2.1 Objectifs pédagogiques et professionnels	12
2.2 Méthode de travail et séquences du stage	12
2.3 Pratiques professionnelles observées	12
2.4 Confidentialité et éthique	13
Chapitre 3 - Contexte, problématique et cahier des charges.....	14
3.1 Contexte de supervision.....	14
3.2 Problématique	14
3.3 Objectifs de la solution	14
3.4 Besoins fonctionnels et non fonctionnels	15
Chapitre 4 - Conception et architecture d'InfraSentinel AI.....	16
4.1 Architecture générale	16
4.2 Flux de données	16
4.3 Modèle multi-tenant et RBAC.....	16
4.4 Architecture locale et conteneurisation	17
Chapitre 5 - Réalisation technique	18
5.1 Agent Windows.....	18
5.2 Backend et données.....	18
5.3 Redis, Celery et traitements asynchrones	19
5.4 Interface web et temps réel	19
5.5 VMware et Hyper-V	21
Chapitre 6 - Intelligence artificielle et supervision proactive.....	22
6.1 Détection d'anomalies	22
6.2 Isolation Forest.....	22
6.3 Analyse temporelle et prédiction de risque	24
6.4 Complémentarité des règles, du ML et des tendances	24
6.5 Limites scientifiques	25
Chapitre 7 - Validation, performance et sécurité	26
7.1 Stratégie de validation	26
7.2 Tests fonctionnels de l'interface web	26
7.3 Tests de charge.....	26
7.4 Télémétrie CPU/GPU et ML	27
7.5 Sécurité.....	27
7.6 Bilan de validation	27
Chapitre 8 - Apports du stage, limites et perspectives.....	29
8.1 Compétences techniques.....	29
8.2 Compétences professionnelles	29
8.3 Difficultés et enseignements	29

8.4 Perspectives	30
Conclusion générale	31
Bibliographie et webographie	32
Annexes	33

Fiche signalétique du stage

Stagiaire	Mehdi Lharti
Établissement	EMSI - École marocaine des sciences de l'ingénieur
Formation	Cycle ingénieur - Informatique et Réseaux
Entreprise	CGI
Lieu	Stage réalisé sur site CGI à Fès - Fès Shore Park
Encadrant professionnel	Nadir Sisah
Encadrant académique	Jaouad Tagnamas
Nature	Stage d'observation
Sujet	Conception d'une solution basée sur l'IA et le machine learning pour la supervision proactive des environnements Windows, VMware et Hyper-V
Période	01/07/2026 – 28/08/2026

Tableau 1 - Fiche signalétique du stage

CHAPITRE 1

Présentation de CGI et cadre professionnel

1.1 Présentation générale de CGI

CGI est une entreprise internationale de services de conseil en technologie de l'information et en management, fondée en 1976. Selon son site institutionnel, le groupe intervient auprès de nombreux secteurs d'activité et s'appuie sur un réseau de plus de 400 sites à l'échelle mondiale. Son positionnement associe conseil, services applicatifs, gestion des infrastructures, transformation numérique et accompagnement des organisations.

L'intérêt d'un stage d'observation au sein d'une telle structure réside dans la possibilité de découvrir une organisation où la technique est indissociable de la qualité de service, de la sécurité, de la relation client, de la gestion des changements et de la documentation. Pour un étudiant en cycle ingénieur, cette immersion permet d'observer comment les connaissances académiques sont transformées en pratiques reproductibles et contrôlées.

1.2 CGI au Maroc et site de Fès

CGI dispose au Maroc de plusieurs implantations, notamment à Casablanca, Rabat et Fès. Le site institutionnel de CGI Maroc indique plus de 1 300 professionnels répartis entre ces implantations et présente les trois sites comme des centres d'excellence mobilisés sur des services applicatifs, la gestion de l'infrastructure et du support, ainsi que des technologies telles que la gestion de données, le CRM, les ERP, les technologies numériques et les environnements historiques.

Le stage a été réalisé dans les locaux de CGI à Fès, au Fès Shore Park. Cette présence locale permet d'inscrire l'expérience dans un environnement professionnel réel tout en restant proche de l'écosystème académique régional. L'observation sur site a également permis de mieux percevoir l'importance des règles d'accès, de confidentialité, de communication et de coordination propres à une entreprise de services numériques.

1.3 Domaines d'activité et environnement professionnel

Domaine	Illustration dans le contexte du stage
Services applicatifs	Conception, développement, intégration, maintenance et validation de solutions logicielles.
Infrastructure et support	Supervision, disponibilité, suivi des performances, traitement des incidents et continuité de service.
Data et IA	Exploitation des données techniques, détection d'anomalies, analyse de tendances et automatisation.
Cybersécurité et qualité	Contrôle d'accès, gestion des secrets, audit, principe du moindre privilège, validation et traçabilité.
Conseil et transformation numérique	Traduction d'un besoin métier ou opérationnel en architecture, processus et solution mesurable.

Tableau 2 - Principaux domaines d'activité observés ou reliés au contexte du stage

Le rapport ne décrit volontairement aucun projet client, aucune donnée interne ni aucun processus confidentiel de CGI. Les éléments présentés concernent uniquement les informations

CHAPITRE 2

Déroulement du stage d'observation

2.1 Objectifs pédagogiques et professionnels

Objectif	Résultat attendu
Découvrir l'environnement professionnel	Comprendre l'organisation générale d'une ESN, la rigueur des échanges et la place de la qualité.
Relier théorie et pratique	Mettre en œuvre architecture logicielle, réseaux, bases de données, systèmes et IA dans un projet cohérent.
Améliorer l'autonomie	Savoir auditer, diagnostiquer, corriger, tester et documenter une solution.
Développer la communication technique	Présenter une architecture, un résultat, une limite et une recommandation de manière claire.
Adopter une démarche d'ingénieur	Mesurer plutôt qu'affirmer, préserver la sécurité et distinguer preuve réelle, simulation et hypothèse.

Tableau 3 - Objectifs du stage

2.2 Méthode de travail et séquences du stage

Le travail a été organisé de manière progressive afin de limiter les régressions et de conserver une trace des décisions. Plutôt que de modifier l'ensemble du projet en une seule étape, la démarche a consisté à établir un état de référence, sécuriser l'environnement, valider chaque composant puis augmenter progressivement le niveau de réalisme des tests.

1. Audit initial du dépôt, de l'architecture, des dépendances, des scripts et des tests.
2. Nettoyage des dépendances cloud inutiles et définition d'un laboratoire local reproductible.
3. Validation de PostgreSQL, Redis, Celery, WebSocket, du backend et du frontend.
4. Validation d'un agent Windows réel et de la remontée de métriques.
5. Renforcement de la détection d'anomalies, des tendances prédictives et des règles temporelles.
6. Campagnes de charge, stress CPU/GPU, sécurité, résilience et sauvegarde/restauration.
7. Reconstruction et amélioration finale de l'interface web, documentation et préparation de la démonstration.

2.3 Pratiques professionnelles observées

Plusieurs principes professionnels ont marqué le déroulement du stage. Le premier est l'importance de la traçabilité : un résultat doit être reproductible et appuyé par un rapport, une commande, un test ou une mesure. Le deuxième est la séparation entre développement et validation : le fait qu'une fonctionnalité existe dans le code ne suffit pas à conclure qu'elle fonctionne dans un environnement réel.

Le stage a également renforcé l'importance de la gestion de configuration et du contrôle de version. Les modifications significatives ont été associées à des points de contrôle Git, des branches dédiées et des rapports de validation. Cette discipline permet de revenir à un état stable, de comparer les évolutions et de limiter l'impact d'une régression.

Enfin, la sécurité et la confidentialité ont été traitées comme des contraintes transversales. Les secrets doivent être injectés par l'environnement, les jetons ne doivent pas être exposés dans

l'interface, les accès doivent être contrôlés côté serveur et les tests doivent être exécutés uniquement sur des équipements autorisés.

2.4 Confidentialité et éthique

Dans une entreprise de services numériques, l'accès aux informations et aux infrastructures doit être proportionné au besoin. Dans le cadre de ce stage, les tests ont été limités aux machines, services et environnements autorisés. Les scénarios de stress ont été bornés dans le temps et accompagnés de seuils de sécurité afin d'éviter une dégradation matérielle ou une perturbation extérieure.

Cette approche est également importante pour la rédaction du rapport. Le document ne présente aucune donnée client, aucun mot de passe, aucun jeton d'agent, aucune adresse interne sensible ni aucune information opérationnelle confidentielle. Les captures intégrées proviennent d'un jeu de démonstration synthétique clairement identifié et ne contiennent ni secret ni donnée client.

CHAPITRE 3

Contexte, problématique et cahier des charges

3.1 Contexte de supervision

Une infrastructure informatique moderne peut réunir postes Windows, serveurs physiques, machines virtuelles, hyperviseurs, services applicatifs, bases de données et composants réseau. Chacun produit des métriques différentes : charge CPU, mémoire, stockage, débit réseau, latence, disponibilité, état des services et événements. Lorsque ces données sont dispersées, il devient difficile d'obtenir une vue cohérente de la santé globale.

La virtualisation renforce cette difficulté. VMware et Hyper-V possèdent leurs propres mécanismes d'administration et de collecte, tandis que Windows expose des informations locales sur le système et le matériel. Une plateforme centralisée doit donc normaliser les données avant de pouvoir appliquer des règles ou des modèles d'analyse communs.

3.2 Problématique

Problématique générale

Comment concevoir une plateforme sécurisée et multi-environnement capable de centraliser des métriques réelles Windows, VMware et Hyper-V, de détecter automatiquement les comportements anormaux et d'identifier des tendances susceptibles d'annoncer un incident avant qu'il ne devienne critique ?

La problématique implique trois niveaux d'analyse. La supervision classique doit d'abord détecter un dépassement de seuil ou une indisponibilité. Le machine learning doit ensuite repérer des combinaisons de métriques inhabituelles qui ne sont pas nécessairement capturées par une règle simple. Enfin, l'analyse temporelle doit mesurer une évolution afin d'estimer qu'une ressource se rapproche d'un état dangereux.

3.3 Objectifs de la solution

- Centraliser l'état des machines et environnements supervisés.
- Collecter des métriques Windows réelles via un agent authentifié.
- Prévoir des connecteurs VMware/vCenter et Hyper-V.
- Normaliser les métriques et les unités avant stockage.
- Déclencher des règles fondées sur un seuil, une durée, un niveau de sévérité et une condition de récupération.
- Détecter des anomalies multidimensionnelles avec Isolation Forest.
- Analyser les tendances et estimer un risque de franchissement de seuil.
- Afficher les alertes, anomalies, prédictions et recommandations dans un tableau de bord en temps réel.
- Garantir le contrôle d'accès par rôles (RBAC), l'isolation multi-client, l'audit, la protection des jetons et la déduplication.
- Valider la solution par tests automatisés, tests de charge, scénarios de résilience et mesures.

3.4 Besoins fonctionnels et non fonctionnels

Catégorie	Besoins principaux
Supervision	Machines, agents, états ONLINE/OFFLINE/UNKNOWN, CPU, RAM, disque, réseau, services et GPU lorsqu'il est disponible.
Virtualisation	Découverte et collecte VMware/vCenter ; collecte Hyper-V via PowerShell et un processus de travail Windows.
IA/ML	Entraînement versionné, inférence, score d'anomalie, stabilité temporelle, tendances et risque.
Alertes	Création, sévérité, déduplication, acquittement, résolution, recommandations et notifications.
Temps réel	WebSocket authentifié, reconnexion, rejeu limité et mise à jour du tableau de bord sans rechargement.
Sécurité	JWT, RBAC, isolation multi-client, jetons d'agents stockés sous forme d'empreinte, CSRF/CSP, secrets hors dépôt, journal d'audit en ajout seulement.
Qualité	Tests automatisés, analyse statique, construction et audit de dépendances, contrôles de santé Docker, sauvegarde/restauration.
Performance	Support d'un nombre croissant d'agents, suivi p50/p95/p99, erreurs et saturation des services.

Tableau 4 - Besoins fonctionnels et non fonctionnels

CHAPITRE 4

Conception et architecture d'InfraSentinel AI

4.1 Architecture générale

L'architecture retenue sépare les sources de données, le traitement serveur, la persistance, l'analyse et la présentation. Cette séparation rend le système plus facile à tester et limite le couplage entre l'agent Windows, les connecteurs de virtualisation et le frontend.

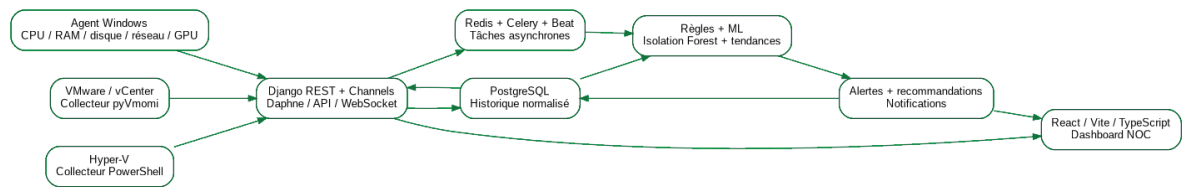


Figure 1 - Architecture générale d'InfraSentinel AI

Le serveur constitue le point central de confiance. Le client ou l'agent ne décide jamais de l'organisation à laquelle une donnée appartient. Les relations entre l'utilisateur, le client, l'environnement, la machine et l'agent sont revalidées côté serveur. Les événements significatifs sont persistés avant leur diffusion en temps réel, ce qui permet un mécanisme de replay contrôlé après reconnexion.

4.2 Flux de données

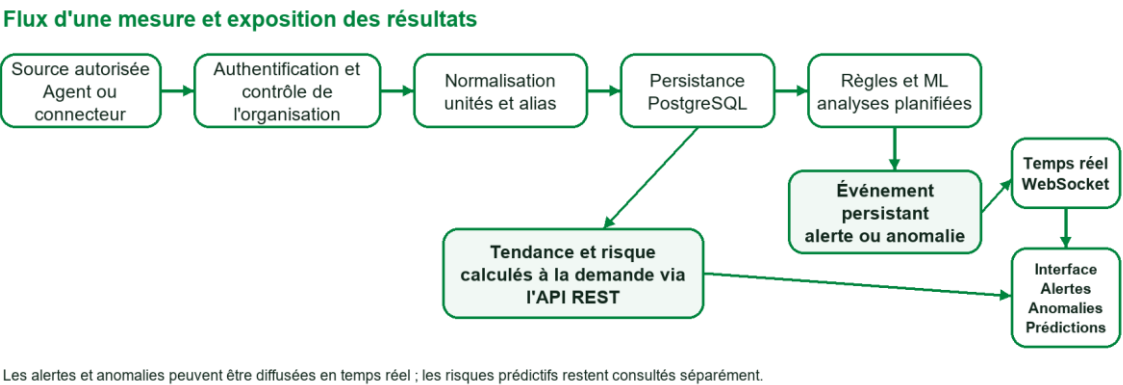


Figure 2 - Flux d'une mesure depuis la source jusqu'au tableau de bord

Une mesure est d'abord authentifiée et rattachée à une source connue. Elle est ensuite normalisée afin de garantir des noms de métriques et des unités cohérents. Après sa persistance dans PostgreSQL, elle peut être utilisée par le moteur de règles, les traitements ML et les agrégations. Les alertes et anomalies persistées peuvent être diffusées en temps réel après validation de la transaction ; les tendances et les risques sont calculés séparément à la demande par l'API REST.

4.3 Modèle multi-tenant et RBAC

InfraSentinel AI a été conçu pour plusieurs rôles. Seul un superutilisateur peut disposer d'une vue transversale sur plusieurs organisations clientes. Un administrateur ordinaire reste limité à son organisation, comme les autres rôles. Cette isolation est appliquée dans les requêtes et les permissions du serveur, et non uniquement dans la barre de navigation de l'interface web.

Rôle	Responsabilité fonctionnelle
------	------------------------------

Rôle	Responsabilité fonctionnelle
ADMIN	Administration de l'organisation, utilisateurs, agents, règles et vue globale autorisée.
SUPERVISOR	Supervision et opérations autorisées sur le périmètre attribué.
TECHNICIAN	Consultation technique en lecture seule des ressources autorisées de son organisation.
CLIENT	Consultation en lecture seule des ressources de son organisation.
VIEWER	Consultation en lecture seule des ressources de son organisation.

Tableau 5 - Rôles et responsabilités applicatives

4.4 Architecture locale et conteneurisation

Après l'abandon des expérimentations dépendantes de fournisseurs cloud, le projet a été consolidé dans un laboratoire local reproductible. Docker Compose orchestre PostgreSQL, Redis, l'API Django/Daphne, Celery Worker, Celery Beat et l'interface web servie par Nginx. Cette approche permet de tester l'ensemble de la chaîne sans coût d'hébergement et de conserver un environnement proche d'un déploiement professionnel.

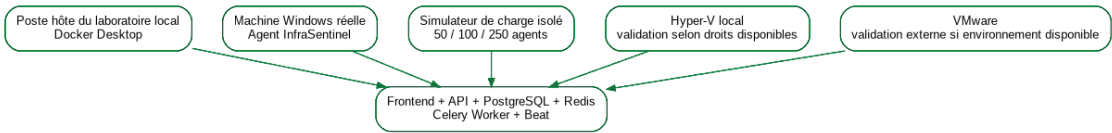


Figure 3 - Architecture du laboratoire local de validation

Les agents simulés sont utilisés uniquement pour la charge. Ils sont isolés des données opérationnelles afin de ne pas être présentés comme de vraies machines. Cette distinction REAL / CONTROLLED_TEST / SIMULATED fait partie des garanties de cohérence de la démonstration.

CHAPITRE 5

Réalisation technique

5.1 Agent Windows

L'agent Windows est écrit en Python. Il collecte des métriques système, envoie périodiquement un signal de présence, ou heartbeat, et transmet les données à l'API après authentification. Les informations couvertes comprennent notamment l'utilisation du processeur, la mémoire, le stockage, les entrées-sorties, le réseau, la durée de fonctionnement, les processus, les services critiques et, lorsqu'une carte NVIDIA est disponible, l'utilisation du GPU, la mémoire vidéo et la température.

L'identité de chaque agent est indépendante. Le serveur stocke l'empreinte du jeton et permet sa révocation ; le renouvellement du jeton passe actuellement par un nouvel enrôlement de l'agent. Il rattache ensuite l'agent à une machine et à une organisation cliente. Sous Windows, la file d'attente locale peut protéger les données en attente au moyen de DPAPI. L'agent refuse les redirections inattendues et impose HTTPS hors de l'environnement local lorsqu'il est configuré en mode sécurisé. Afin de préserver la confidentialité de la machine réelle, les figures 4 à 10 et 12 utilisent le jeu de démonstration local PFE25, dont les données sont explicitement signalées comme synthétiques.

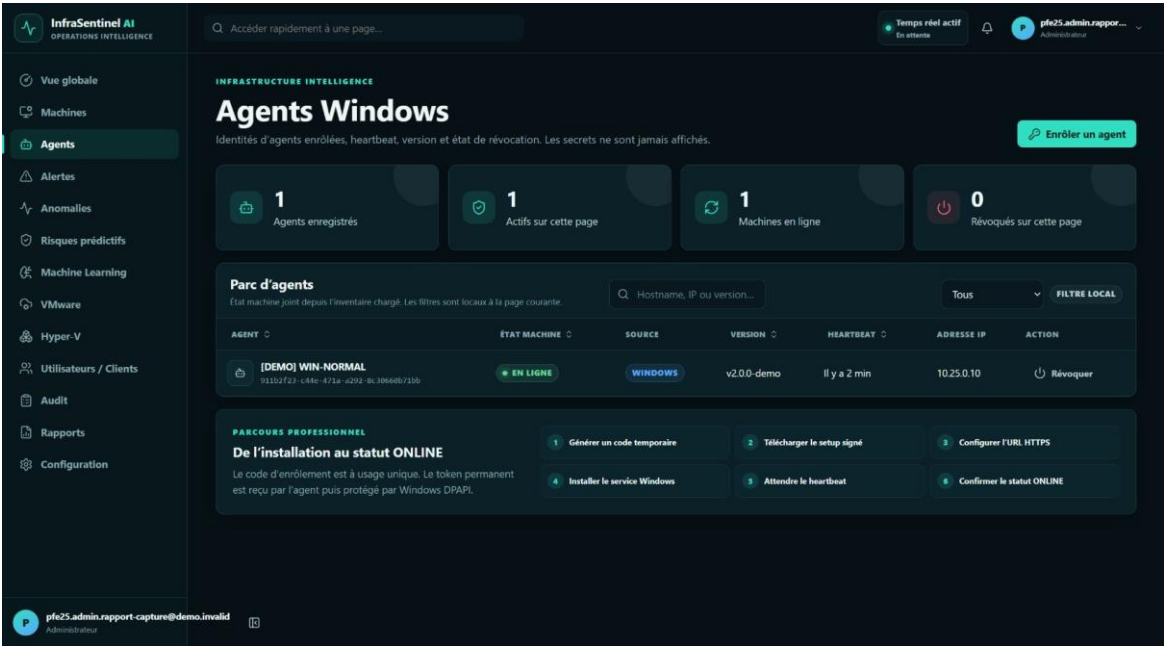


Figure 4 - Agent Windows de la démonstration synthétique PFE25

5.2 Backend et données

Le backend repose sur Django 6, Django REST Framework, SimpleJWT, Channels et Daphne. PostgreSQL constitue la base de référence. Les domaines métiers sont séparés entre comptes, inventaire, métriques, monitoring, moteur ML, intégrations, notifications, temps réel et tâches asynchrones. Cette structuration facilite les tests et la maîtrise des responsabilités.

Composant	Technologie / rôle
Backend API	Django 6.0.8, Django REST Framework 3.17.1, SimpleJWT
Temps réel	Django Channels 4.3.2, Daphne 4.2.3, channels-redis

Composant	Technologie / rôle
Base de données	PostgreSQL 17 avec psycopg et un pool de connexions
Asynchrone	Celery 5.6 + Redis, Celery Beat
Apprentissage automatique	scikit-learn 1.9, Isolation Forest, ingénierie des variables et tendances
VMware	pyVmoi 9
Hyper-V	PowerShell exécuté par un processus Windows disposant des permissions adaptées
Interface web	React 19, TypeScript 5.9, Vite, TanStack Query, Recharts, Axios
Conteneurisation	Docker Compose, Nginx; Caddy pour TLS dans la configuration complémentaire de production

Tableau 6 - Technologies principales

5.3 Redis, Celery et traitements asynchrones

Redis est utilisé pour la communication asynchrone et le canal de diffusion temps réel. Celery exécute les traitements qui ne doivent pas bloquer une requête HTTP : collectes externes, évaluations de règles, analyses ML, notifications, agrégations et rapports. Celery Beat publie les tâches planifiées.

L'idempotence est importante pour éviter qu'une tâche rejouée crée plusieurs alertes ou plusieurs métriques. Le projet associe des clés métiers aux opérations critiques et conserve un historique des exécutions. La campagne de résilience a également testé l'arrêt puis le redémarrage de Redis, du worker, de Beat, de l'API et de PostgreSQL.

5.4 Interface web et temps réel

L'interface web finale a été reconstruite avec React, Vite et TypeScript, selon une architecture par domaines, avec un client API centralisé, TanStack Query pour le cache et Recharts pour les visualisations. Elle reprend les codes d'un outil NOC : état global, machines nécessitant une attention, alertes critiques, anomalies ML, risques prédictifs et activité récente.

Le WebSocket permet de mettre à jour les états importants sans recharger la page. La logique gère l'obtention d'un ticket, la négociation de connexion, la reconnexion, le jeu limité et la déduplication. Le statut de la connexion en temps réel est visible dans l'interface afin de distinguer un écran à jour d'une connexion interrompue.

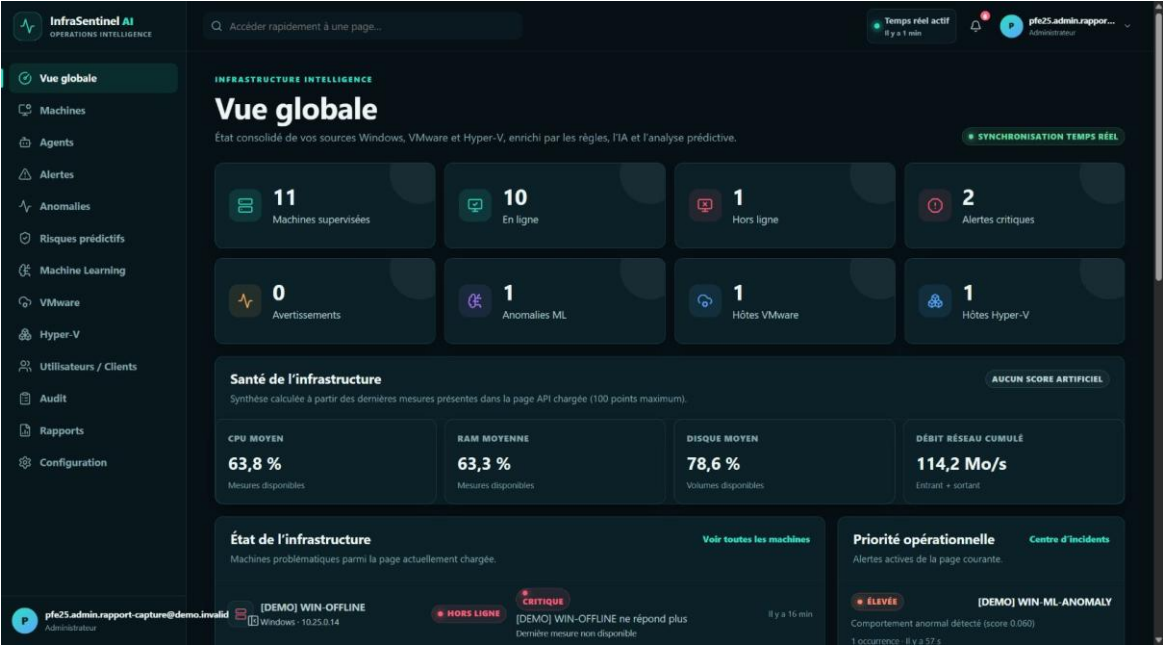


Figure 5 - Vue globale de la démonstration synthétique PFE25

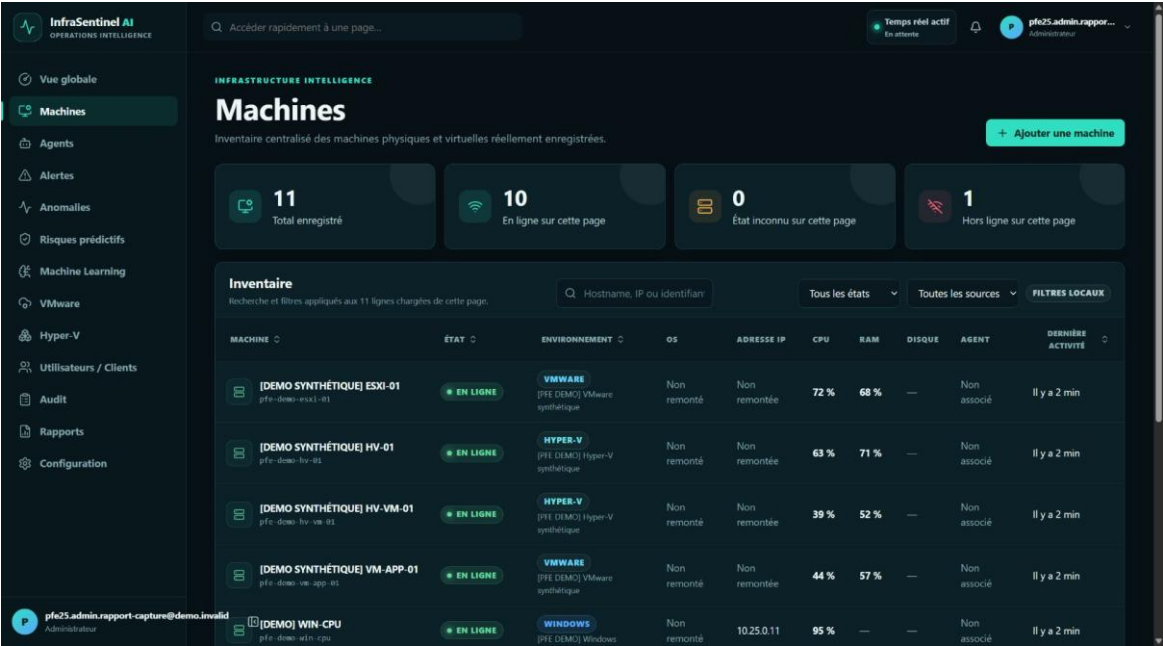


Figure 6 - Inventaire des machines de la démonstration synthétique PFE25

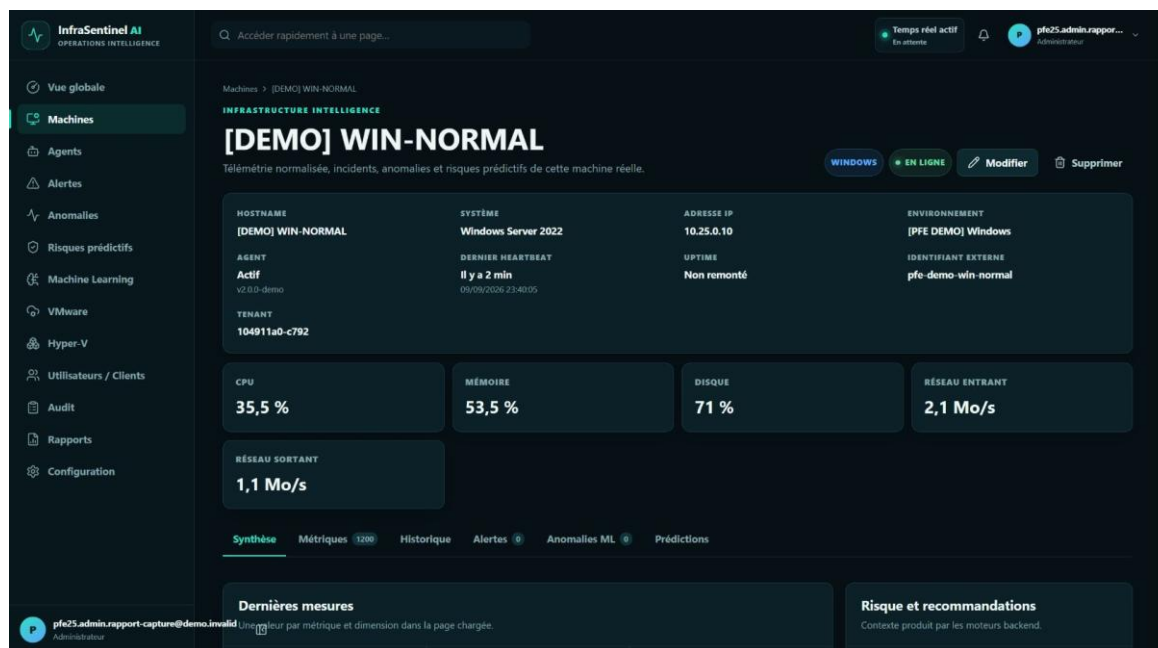


Figure 7 - Fiche d'une machine Windows de la démonstration synthétique PFE25

5.5 VMware et Hyper-V

Deux connecteurs complètent l'agent Windows. La partie VMware s'appuie sur pyVmomi afin de découvrir les hôtes et machines virtuelles vSphere et de normaliser leurs métriques. La partie Hyper-V repose sur PowerShell et nécessite un worker Windows disposant du module Hyper-V ainsi que des permissions adaptées.

Ces composants sont présents, testés avec des doublures et intégrés aux API et à l'interface web. Toutefois, le rapport conserve une distinction stricte entre l'implémentation et la preuve réelle : la campagne locale ne disposait pas d'un vCenter autorisé et l'appel à Get-VM a échoué faute de privilèges suffisants. Les validations VMware et Hyper-V réelles restent donc à compléter sur une infrastructure autorisée.

CHAPITRE 6

Intelligence artificielle et supervision proactive

6.1 Détection d'anomalies

La valeur ajoutée du sujet ne réside pas uniquement dans l'affichage de métriques. Une plateforme proactive doit repérer qu'un comportement s'éloigne de la normale et fournir une information suffisamment tôt pour permettre une action préventive. Pour cela, InfraSentinel AI construit des fenêtres de métriques historiques et applique un modèle non supervisé.

6.2 Isolation Forest

Isolation Forest est adapté à la détection d'anomalies lorsque les exemples explicitement étiquetés « panne » sont rares. Le principe consiste à isoler plus rapidement les observations atypiques que les observations appartenant à une zone dense du comportement habituel. Le modèle produit donc un score d'anomalie qu'il faut interpréter dans le contexte des variables utilisées et du seuil de décision.

Le paramètre de contamination ne représente pas une probabilité de panne. Il sert à calibrer la proportion attendue d'observations atypiques lors de l'entraînement. La qualité du modèle dépend surtout de la représentativité du comportement normal, du volume de fenêtres, du choix des variables et de la stabilité de la politique d'alerte.

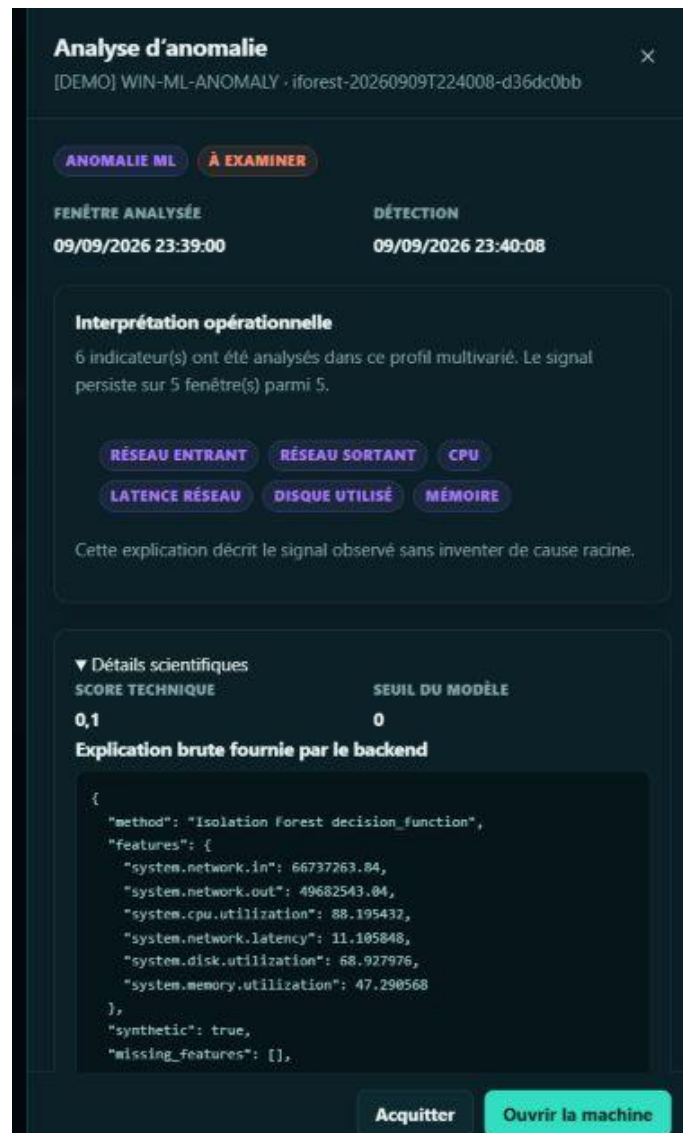


Figure 8 - Analyse d'une anomalie de la démonstration synthétique PFE25

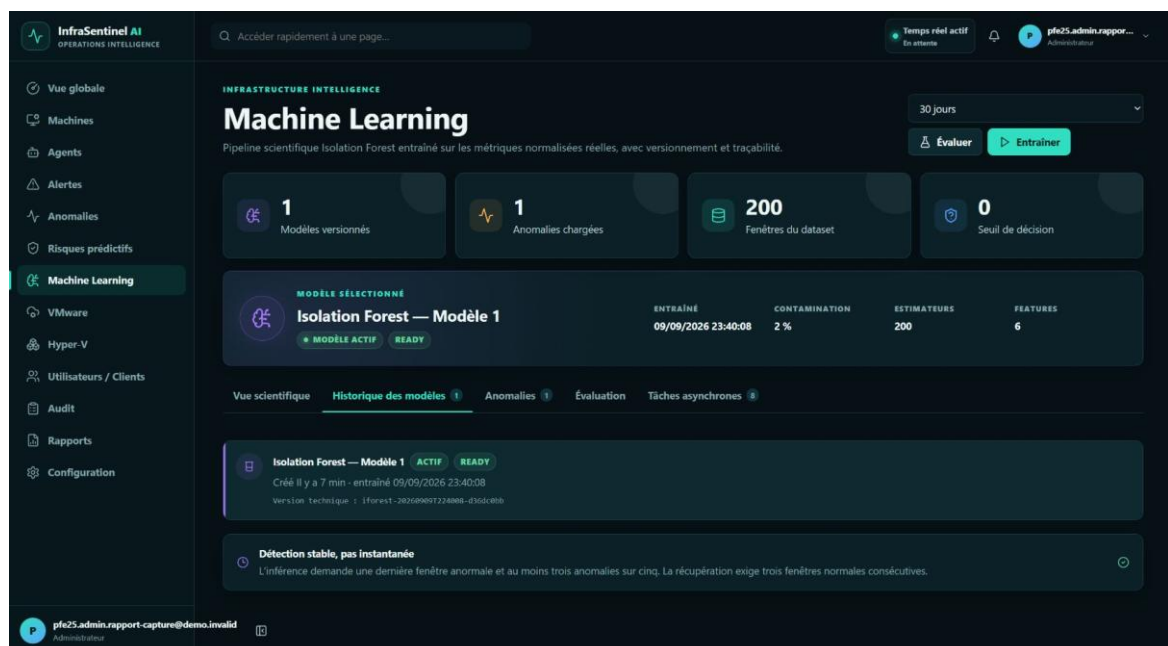


Figure 9 - Historique du modèle Isolation Forest de la démonstration synthétique PFE25

6.3 Analyse temporelle et prédiction de risque

Isolation Forest détecte principalement une situation inhabituelle ; il ne prédit pas à lui seul l'heure exacte d'une panne. Pour rendre la supervision réellement proactive, InfraSentinel complète l'inférence par une analyse temporelle : pente, vitesse d'évolution, persistance et estimation du temps avant franchissement d'un seuil lorsque le calcul est pertinent.

Une série de valeurs de RAM passant par exemple de 52 % à 58 %, 64 %, 71 % puis 79 % peut produire une tendance croissante avant que la règle critique ne se déclenche. Le système peut alors présenter un risque élevé et une estimation prudente du franchissement potentiel, tout en affichant « historique insuffisant » lorsqu'il ne dispose pas de données suffisantes.

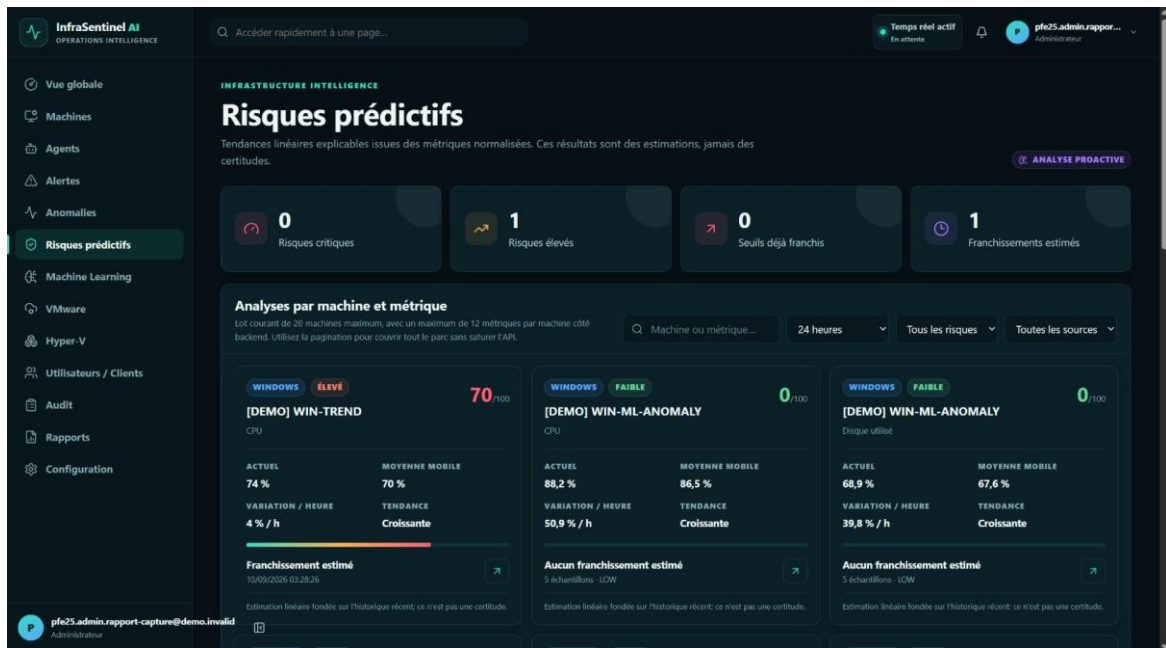


Figure 10 - Analyse prédictive de la démonstration synthétique PFE25

6.4 Complémentarité des règles, du ML et des tendances

Mécanismes complémentaires de supervision proactive

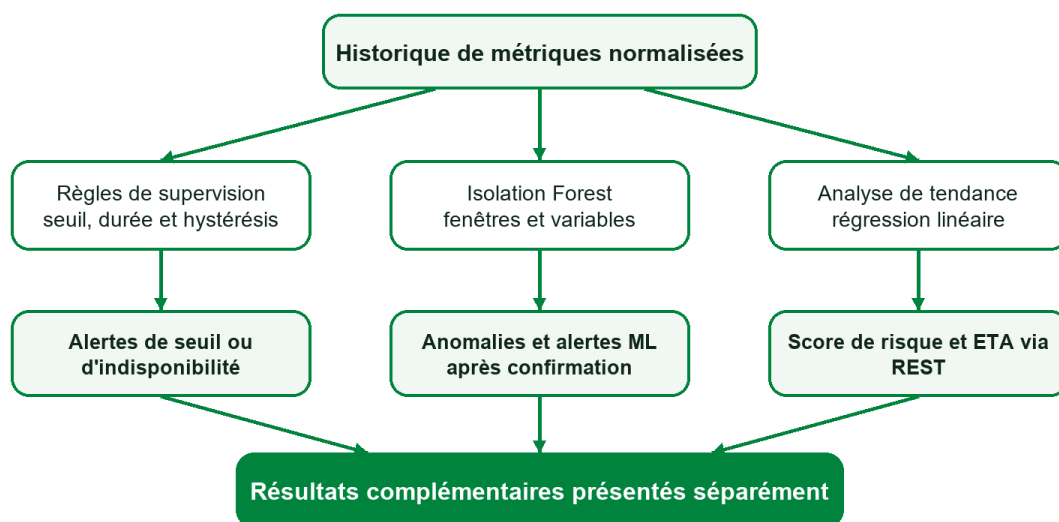


Figure 11 - Vue conceptuelle des mécanismes de supervision proactive

Les trois mécanismes sont complémentaires, mais restent exposés séparément. Les règles déclenchent des alertes de seuil ou d'indisponibilité. Isolation Forest crée des anomalies et des alertes ML après confirmation temporelle. L'analyse de tendance calcule séparément un score de

risque et, lorsque cela est pertinent, une estimation linéaire du franchissement d'un seuil. Le code actuel n'agrège pas ces trois signaux dans un score unique.

Pour réduire les faux positifs, le moteur ML ne fait pas nécessairement remonter le niveau d'une alerte à partir d'une seule fenêtre. Des critères de persistance et d'hystérésis imposent plusieurs fenêtres anormales avant toute escalade et plusieurs fenêtres normales avant la résolution. Cette stabilité temporelle est essentielle dans un système de supervision réel.

6.5 Limites scientifiques

La validation scientifique d'un modèle d'anomalies est plus difficile qu'un simple test logiciel. Pour mesurer précisément la précision, le rappel, le score F1, les faux positifs et les faux négatifs, il faut disposer d'une vérité terrain correctement étiquetée. Les campagnes du stage ont renforcé le nombre de fenêtres normales et vérifié des anomalies contrôlées, mais elles ne constituent pas encore un jeu de données industriel à long terme.

La bonne pratique retenue est donc de présenter séparément : une détection sur métriques réelles, un scénario contrôlé servant à reproduire une anomalie et les limites restantes. Cette transparence est préférable à l'affichage de scores artificiellement parfaits.

CHAPITRE 7

Validation, performance et sécurité

7.1 Stratégie de validation

La validation a été organisée selon plusieurs niveaux : tests unitaires, tests d'intégration, construction, contrôles de santé Docker, test d'agent réel, tests réseau et WebSocket, scénarios de charge, stress matériel contrôlé, résilience et audit de sécurité. Les résultats sont classés en PASS, PARTIAL, FAIL ou NOT TESTED afin d'éviter de confondre l'existence d'un composant et sa validation dans une infrastructure réelle.

7.2 Tests fonctionnels de l'interface web

Une réexécution sur la version actuelle du dépôt a porté sur 64 tests Vitest répartis sur 17 fichiers : 63 ont réussi et 1 a échoué dans le test de déconnexion d'AppShell. Les vérifications TypeScript, ESLint et Prettier ainsi que la construction locale ont réussi. L'audit des dépendances de production (npm audit --omit=dev) ne signale aucune vulnérabilité connue. L'audit complet signale toutefois trois vulnérabilités dans les dépendances de développement, dont une élevée, qui restent à traiter ou à documenter. L'interface web est donc partiellement validée et ne doit pas être présentée comme intégralement au vert.

La validation a couvert les routes principales, les états de chargement et d'erreur, la pagination prédictive, la déconnexion sur mobile, la navigation au clavier, le cache multi-client, le WebSocket, les unités lisibles et la politique CSP de même origine. L'interface web reste adaptée à une démonstration PFE, sous réserve de corriger le test de déconnexion d'AppShell et de poursuivre certains audits spécialisés et certaines intégrations externes.

7.3 Tests de charge

Profil	Agents	Débit	p95	Erreurs	Connexions PG max
Simulé, intervalle 30 s	50	2,80 req/s	71,6 ms	0 %	16
Simulé, intervalle 30 s	100	5,60 req/s	76,2 ms	0 %	16
Simulé, intervalle 30 s	250	13,74 req/s	98,1 ms	0 %	16
Simulé accéléré, intervalle 1 s	25	25,01 req/s	113,1 ms	0 %	17
Simulé accéléré, intervalle 1 s	50	45,52 req/s	1 334,6 ms	0 %	26
Simulé accéléré, intervalle 1 s	100	45,16 req/s	2 604,1 ms	0 %	26

Tableau 7 - Résultats principaux de performance après remédiation

Dans un essai local en boucle locale, mesuré pendant 30 secondes après 10 secondes de préchauffage, 250 agents simulés à cadence réaliste ont produit 0 % d'erreur et un p95 de 98,1 ms. L'essai utilisait une instance Daphne dédiée avec un quota d'agents relevé. Le scénario accéléré montre que la latence augmente lorsque la cadence devient beaucoup plus agressive. Ces mesures valident des scénarios courts de laboratoire, mais ne démontrent ni l'endurance ni une capacité de production.

7.4 Télémétrie CPU/GPU et ML

Les tests matériels ont été bornés par des seuils de sécurité. La télémétrie GPU de la machine Windows réelle a été validée avec une NVIDIA GeForce RTX 5070 Laptop GPU : la campagne a observé environ 65 % d'utilisation, 74 °C au maximum et plus de 4,4 Go de VRAM dans un scénario contrôlé. L'agent a transmis les métriques correspondantes au backend.

La variable GPU a ensuite été intégrée au schéma ML de manière optionnelle afin de ne pas casser les machines sans GPU. La campagne a confirmé la collecte et l'utilisation de cette variable, mais n'a pas fabriqué une anomalie GPU lorsque le score réel restait sous le seuil. Ce résultat PARTIAL est méthodologiquement correct : l'absence d'anomalie n'est pas remplacée par une donnée fictive.

7.5 Sécurité

La sécurité du projet couvre plusieurs couches : authentification JWT, jeton de rafraîchissement HttpOnly côté navigateur, protection CSRF, RBAC, isolation multi-client, jetons d'agents opaques stockés sous forme d'empreinte, journal d'audit PostgreSQL en ajout seulement, CSP, limitation de la taille des charges utiles, neutralisation des données sensibles dans les messages d'erreur, conteneurs non privilégiés et séparation des secrets. PostgreSQL et Redis ne doivent pas être exposés directement au réseau utilisateur.

Les campagnes de sécurité ont également vérifié les dépendances Python et npm, le profil de déploiement Django, les permissions des conteneurs et la protection des communications de l'agent. Les risques résiduels documentés concernent notamment l'absence de MFA, la nécessité de filtrer les flux sortants des connecteurs internes et certaines validations de production qui dépendent d'une infrastructure extérieure.

7.6 Bilan de validation

Domaine	État	Commentaire
Interface web	PARTIAL	17 fichiers et 64 tests : 63 réussis, 1 échec sur le test de déconnexion d'AppShell ; vérification des types, lint, formatage et construction locale réussis.
Backend / API	PASS local	Health, API, persistance, règles, ML et sécurité couverts par la suite de tests.
PostgreSQL / Redis / Celery	PASS local	Services sains, résilience et sauvegarde/restauration validées localement.
WebSocket	PASS	Ticket HTTP 200, négociation de connexion 101, reconnexion et rejeu et sécurité.
Agent Windows réel	PASS	Enrôlement, heartbeat, métriques et reconnexion validés.

Domaine	État	Commentaire
Performance	PASS/PARTIAL	250 agents simulés à cadence réaliste sans erreur ; latence élevée en cadence accélérée.
ML anomalies	PARTIAL	Détection réelle et stabilité améliorée ; vérité terrain industrielle à renforcer.
Prédicatif	PARTIAL	Tendances et ETA implémentées ; validation à long terme à poursuivre.
VMware réel	NOT TESTED	Connecteur présent mais aucun vCenter autorisé disponible pendant la campagne.
Hyper-V réel	NOT TESTED	Connecteur présent ; permissions Get-VM insuffisantes lors de la campagne.
SMTP externe	NOT TESTED	Chaîne applicative testée, livraison externe non qualifiée.
Windows Service	NOT TESTED / partiel	Code et scripts présents ; campagne finale à exécuter avec élévation et redémarrage.

Tableau 8 - Synthèse des validations techniques

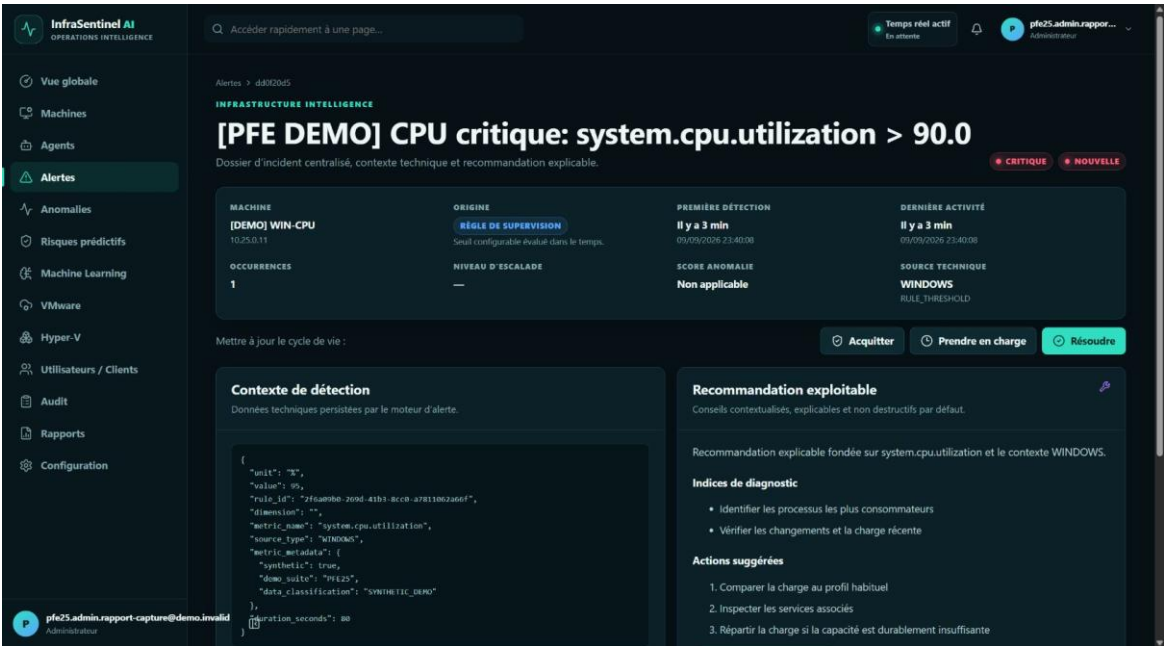


Figure 12 - Alerte CPU et recommandation de la démonstration synthétique PFE25

CHAPITRE 8

Apports du stage, limites et perspectives

8.1 Compétences techniques

Le projet a mobilisé des compétences transversales couvrant les systèmes, le développement, les réseaux, les bases de données, le DevOps et l'IA. La mise en place de Docker, PostgreSQL, Redis et Celery a permis de comprendre la différence entre une application monolithique simple et une plateforme répartie en plusieurs services. Les tests de charge ont également montré l'importance du dimensionnement, du pool de connexions et de la mesure des latences.

- Architecture Django REST et séparation des domaines métiers.
- Interface web React/TypeScript et conception d'une interface de supervision NOC.
- Collecte Windows réelle, métriques système et GPU NVIDIA.
- PostgreSQL, Redis, Celery, Beat et traitements asynchrones.
- WebSocket, reconnexion, rejeu et déduplication d'événements.
- Docker Compose, contrôles de santé, sauvegarde et restauration.
- Isolation Forest, ingénierie des variables, tendances et limites d'un modèle non supervisé.
- RBAC, isolation multi-client, gestion des secrets, audit et sécurité applicative.
- Tests unitaires, intégration, performance, stress contrôlé et validation reproductible.

8.2 Compétences professionnelles

L'apport du stage ne se limite pas aux outils. La principale évolution concerne la manière d'aborder un problème : commencer par observer et auditer, identifier les hypothèses, distinguer les exigences des préférences, construire une preuve, documenter une limite et conserver un état stable. Cette démarche est directement transférable à de futurs projets en entreprise.

Le stage a également renforcé la communication technique. Un rapport utile ne doit pas seulement indiquer qu'un test a échoué ; il doit préciser le contexte, la cause identifiée, l'impact, la correction et le résultat après remédiation. De même, une démonstration doit préciser l'origine et le caractère réel, contrôlé ou synthétique des données, et annoncer explicitement lorsqu'une intégration externe n'a pas pu être validée.

8.3 Difficultés et enseignements

Plusieurs difficultés ont joué un rôle formateur. L'hébergement cloud gratuit s'est révélé peu adapté à une architecture nécessitant PostgreSQL, Redis, Celery et des connexions WebSocket persistantes ; le projet a donc été recentré sur un laboratoire local complet. La saturation initiale de PostgreSQL à 100 connexions a montré qu'une erreur sous charge peut provenir d'une ressource partagée plutôt que du code du point d'entrée API lui-même.

La partie ML a également montré qu'un modèle qui produit des anomalies n'est pas automatiquement un bon modèle. Avec peu de fenêtres d'entraînement, les faux positifs deviennent difficiles à interpréter. L'augmentation de l'historique, la stabilité temporelle et l'hystérésis ont donc été plus importantes que l'ajout d'un nouvel algorithme plus complexe.

Enfin, les limites VMware et Hyper-V ont rappelé qu'une intégration réelle dépend autant des droits et de l'environnement que du code. Un connecteur correctement testé avec des doublures ne doit pas être présenté comme validé sur un hyperviseur réel tant qu'il n'a pas été testé dans un environnement réel avec les autorisations nécessaires.

8.4 Perspectives

Perspective	Objectif
Hyper-V réel	Exécuter Get-VM avec un compte autorisé, collecter plusieurs VM et valider le cycle normal/hors ligne/récupération.
VMware réel	Connecter un ESXi/vCenter de laboratoire, valider découverte, datastore, état d'alimentation et métriques.
Windows Service	Installer le service, tester démarrage automatique, redémarrage, reconnexion et désinstallation.
ML à long terme	Collecter davantage de données normales, introduire un protocole de labellisation et mesurer précision/rappel/F1.
Prédiction	Valider sur séries réelles longues l'estimation de franchissement de seuil et mesurer l'erreur temporelle.
Scalabilité	Profilier le backend et la base de données, tester des cadences plus importantes et dimensionner un gestionnaire de connexions si nécessaire.
Production	TLS, domaine, proxy inverse, gestion de secrets, CI/CD et supervision de la plateforme elle-même.
Accessibilité	Compléter l'audit au lecteur d'écran et la validation multirôle dans plusieurs navigateurs.

Tableau 9 - Limites et perspectives

Conclusion générale

Ce stage d'observation professionnelle réalisé sur site au sein de CGI a permis de relier les enseignements du cycle ingénieur à une démarche de projet plus rigoureuse. L'expérience a donné une vision concrète des exigences d'un environnement professionnel : comprendre le besoin, sécuriser les accès, structurer l'architecture, mesurer les performances, gérer les erreurs, documenter les décisions et ne pas confondre une fonctionnalité développée avec une fonctionnalité réellement validée.

InfraSentinel AI constitue le principal support technique de cette expérience. La plateforme centralise la supervision Windows, prévoit des connecteurs VMware et Hyper-V, utilise PostgreSQL, Redis, Celery et WebSocket, et propose une interface web en temps réel. Elle présente de façon complémentaire les règles de supervision, la détection d'anomalies par Isolation Forest et l'analyse de tendances afin de contribuer à une supervision proactive.

Les résultats locaux sont encourageants : agent Windows réel, télémétrie CPU/GPU, environnement Docker sain, communications en temps réel fonctionnelles et essai court de 250 agents simulés à cadence réaliste sans erreur. L'interface web est couverte par 64 tests Vitest, dont 63 réussissent sur la version actuelle du dépôt, sans suite E2E Playwright intégrée au dépôt. En parallèle, les limites ont été documentées avec transparence : VMware et Hyper-V réels, certains scénarios du service Windows, SMTP externe et validation scientifique sur une longue durée restent à compléter.

Au-delà du résultat logiciel, le stage a surtout développé une méthode : observer, mesurer, corriger, vérifier et expliquer. Cette méthode représente l'un des acquis les plus importants de l'immersion professionnelle et constituera une base utile pour de futurs projets d'infrastructure, de cybersécurité, de DevOps et d'intelligence artificielle.

Annexes

Annexe A - Commandes principales de lancement local

Les commandes ci-dessous sont indicatives et doivent être exécutées depuis la racine du dépôt, sans afficher ni recopier de secrets dans le rapport.

```
git status
docker compose build
docker compose up -d
docker compose ps
python manage.py check
python manage.py migrate
npm run typecheck
npm run lint
npm test
npm run build
```

Annexe B - Synthèse des résultats techniques

Indicateur	Résultat de référence
Interface web	17 fichiers / 64 tests : 63 réussis, 1 échec, 0 ignoré
TypeScript / ESLint / Prettier	PASS
Construction de l'interface web	PASS local et Docker
npm audit	Production : 0 vulnérabilité connue ; audit complet : 3 vulnérabilités de développement, dont 1 élevée
WebSocket	Ticket HTTP 200, négociation de connexion 101, reconnexion et rejeu couverts
Charge réaliste	250 agents simulés à cadence réaliste, intervalle de 30 s, 0 % d'erreur, p95 98,1 ms
PostgreSQL après remédiation	26 connexions max en charge accélérée étudiée, 0 erreur aux paliers rejoués
GPU réel	Environ 65 % utilisation max, 74 °C max, plus de 4,4 Go de VRAM dans le scénario contrôlé
Limites externes	VMware réel, Hyper-V réel, SMTP externe et certains scénarios du service Windows et de redémarrage non qualifiés

Annexe C - Traçabilité des captures intégrées

- Vue globale : indicateurs agrégés du jeu de démonstration synthétique PFE25.
- Machines : inventaire de onze machines synthétiques, avec leurs états et métriques principales.
- Fiche machine : métriques CPU, mémoire, disque et réseau d'une machine Windows de démonstration.

- Alertes : alerte CPU issue d'une règle de supervision, avec une recommandation non destructive.
- Anomalies ML : anomalie synthétique avec indicateurs analysés, score et détails scientifiques.
- Modèle ML : historique du modèle Isolation Forest utilisé par la démonstration.
- Prédictions : tendance, niveau de risque, valeur actuelle et estimation du franchissement.
- Agents : agent Windows synthétique ; aucun jeton, secret ou chemin privé n'est visible.

Annexe D - Note sur la validation réelle et contrôlée

Règle de présentation au jury

Une preuve simulée sert à valider la scalabilité ou la reproductibilité d'un scénario. Elle ne remplace pas une preuve d'intégration réelle. Le rapport et la démonstration doivent conserver les libellés REAL PASS, CONTROLLED TEST PASS, SIMULATED LOAD TEST PASS et NOT TESTED lorsque ces distinctions sont pertinentes.

Fin du rapport